

(use "git add <file>..." to include in what will be committed)

④ untracked_file.py

In this working copy, there's a *new_file.py* ❶, which has recently been added to the repo and is therefore in the staged state. There are also two tracked files, *staged_file.py* ❷ and *modified_file.py* ❸, which are in the staged and modified states, respectively. Then there's an untracked file named *untracked_file.py* ❹. The output of `git status` also has reminders for the Git commands that move the files to other states.

Why Stage Files?

You might wonder what the point of the staged state is. Why not just go between *modified* and *committed* without staging files? Dealing with the staging area is full of thorny special cases and a large source of confusion for Git beginners. For instance, a file can be modified after it has been staged, leading to files existing in both the modified and staged states, as described in the previous section. Technically, the staging area doesn't contain files so much as changes, because parts of a single modified file can be staged and other parts unstaged. Cases like these are why Git has a reputation for being complex, and many sources of information on how Git works are often imprecise at best and misleading at worst.

But we can avoid most of this complexity. In this chapter, I recommend avoiding it by using the `git commit -am` command to stage and commit modified files in a single step. This way they'll move directly from the modified state to the clean state. Also, I recommend always immediately committing files after adding, renaming, or removing them in your repo. Additionally, using GUI Git tools (explained later) rather than the command line can help you avoid these tricky cases.

Creating a Git Repo on Your Computer

Git is a *distributed version control system*, which means it stores all of its snapshots and repo metadata locally on your computer in a folder named *.git*. Unlike a centralized version control system, Git doesn't need to